

DEPARTMENT OF COMPUTER SCIENCE (P.E)

*Human-Computer Interaction (HCI) & Computer Graphics (CG)***LAB REPORT 1****Display Density Metrics & Image Array Mechanics in NumPy**

Student Details	
Name	Khadija intikhab
Roll No / ID	2k24/CSE/80
Course	Human-Computer Interaction & Computer Graphics
Submitted To	Sir rajesh kumar
Date	September 11, 2026
Tool Used	Online GDB Compiler (Python 3)

Lab Objectives

- Compute spatial display metrics including resolution, total pixels, aspect ratio, and DPI/PPI.
- Set up a Python computing environment for visual graphics tasks.
- Construct, inspect, and manipulate 3D image arrays ($H \times W \times C$) using NumPy.
- Perform color channel extraction, spatial downsampling, and array striding operations.

1. Task 1: Display Pixel Density (PPI/DPI) Calculator

Objective: Write a Python script that calculates screen parameters and classifies display density based on physical screen attributes.

Source Code

```
import math

def classify_density(dpi):
    if dpi < 100:
        return "Low Density (Standard Monitor)"
    elif dpi <= 200:
        return "Medium Density (HD Display)"
    else:
        return "High Density (Retina / Mobile)"

def main():
    w_px = int(input("Enter horizontal resolution (pixels): "))
    h_px = int(input("Enter vertical resolution (pixels): "))
    d_inches = float(input("Enter physical diagonal size (inches): "))

    total_pixels = w_px * h_px

    divisor = math.gcd(w_px, h_px)
    aspect_w = w_px // divisor
    aspect_h = h_px // divisor

    diagonal_px = math.sqrt(w_px ** 2 + h_px ** 2)
    dpi = diagonal_px / d_inches

    category = classify_density(dpi)

    print("\n--- DISPLAY METRICS ANALYSIS ---")
    print(f"Total Pixel Count : {total_pixels:,} pixels")
    print(f"Aspect Ratio       : {aspect_w}:{aspect_h}")
    print(f"Calculated DPI      : {dpi:.2f} DPI")
    print(f"Density Category    : {category}")

if __name__ == "__main__":
    main()
```

Execution on Online GDB

```

12 w_px = int(input("Enter horizontal resolution (pixels): "))
13 h_px = int(input("Enter vertical resolution (pixels): "))
14 d_inches = float(input("Enter physical diagonal size (inches): "))
15
16 total_pixels = w_px * h_px
17
18 divisor = math.gcd(w_px, h_px)
19 aspect_w = w_px // divisor
20 aspect_h = h_px // divisor
21
22 diagonal_px = math.sqrt(w_px ** 2 + h_px ** 2)
23 dpi = diagonal_px / d_inches
24
25 category = classify_density(dpi)
26
27 print("\n--- DISPLAY METRICS ANALYSIS ---")
28 print(f"Total Pixel Count : {total_pixels:,} pixels")
29 print(f"Aspect Ratio       : {aspect_w}:{aspect_h}")

```

Enter horizontal resolution (pixels): 20
Enter vertical resolution (pixels): 50
Enter physical diagonal size (inches): 43

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 1,000 pixels
Aspect Ratio : 2:5
Calculated DPI : 1.25 DPI
Density Category : Low Density (Standard Monitor)

...Program finished with exit code 0
Press ENTER to exit console.

Fig 1.1 — Task 1 running on Online GDB (test input: 20 × 50 px, 43").

Reference Results (required by the checklist)

Device	Resolution	Diagonal	Total Pixels	Aspect Ratio	DPI	Category
Desktop Monitor	1920 × 1080	24"	2,073,600	16:9	91.79	Low Density
Smartphone	1170 × 2532	6.1"	2,962,440	195:422	457.25	High Density

Observation: The program correctly reads three integer/float inputs, computes the total pixel count as the product of width and height, reduces the resolution ratio to its simplest form using `math.gcd()`, and derives DPI from the Pythagorean diagonal-pixel-count divided by the physical screen size. The 20×50 px / 43" test case correctly falls into the 'Low Density' bracket since 1.25 DPI is far below the 100 DPI threshold — this is expected, since the resolution used was arbitrarily small for a quick sanity test rather than a realistic screen.

2. Task 2: Environment Setup & Synthetic Image Matrix Creation

Objective: Install Python graphics packages and construct a 3D image matrix from scratch.

Source Code

```

import numpy as np

def main():
    H, W, C = 300, 400, 3
    img = np.zeros((H, W, C), dtype=np.uint8)

    mid_h = H // 2
    mid_w = W // 2

    img[0:mid_h, 0:mid_w] = [255, 0, 0]
    img[0:mid_h, mid_w:W] = [0, 255, 0]
    img[mid_h:H, 0:mid_w] = [0, 0, 255]
    img[mid_h:H, mid_w:W] = [255, 255, 255]

```

```

print("--- SYNTHETIC MATRIX METRICS ---")
print(f"Array Shape (H, W, C) : {img.shape}")
print(f"Data Type           : {img.dtype}")
print(f"Total Elements       : {img.size:,} values")
print(f"Memory Footprint      : {img.nbytes:,} bytes ({img.nbytes/1024:.2f} KB)")

try:
    from PIL import Image
    Image.fromarray(img).save("synthetic_quadrants.png")
    print("\nSaved visual output as 'synthetic_quadrants.png'")
except ImportError:
    print("\n(Pillow not installed - skipping image save)")

if __name__ == "__main__":
    main()

```

Execution on Online GDB

The screenshot displays the OnlineGDB web interface. On the left is a sidebar with navigation links like 'My Projects', 'Classroom', and 'Login'. The main area shows a Python script being executed. The script creates a synthetic matrix with dimensions (300, 400, 3) and prints its metrics. The console output at the bottom shows the following results:

```

--- SYNTHETIC MATRIX METRICS ---
Array Shape (H, W, C) : (300, 400, 3)
Data Type           : uint8
Total Elements       : 360,000 values
Memory Footprint      : 360,000 bytes (351.56 KB)

Saved visual output as 'synthetic_quadrants.png'

...Program finished with exit code 0
Press ENTER to exit console.

```

Fig 2.1 — Console output showing array shape, dtype, element count, and memory footprint.

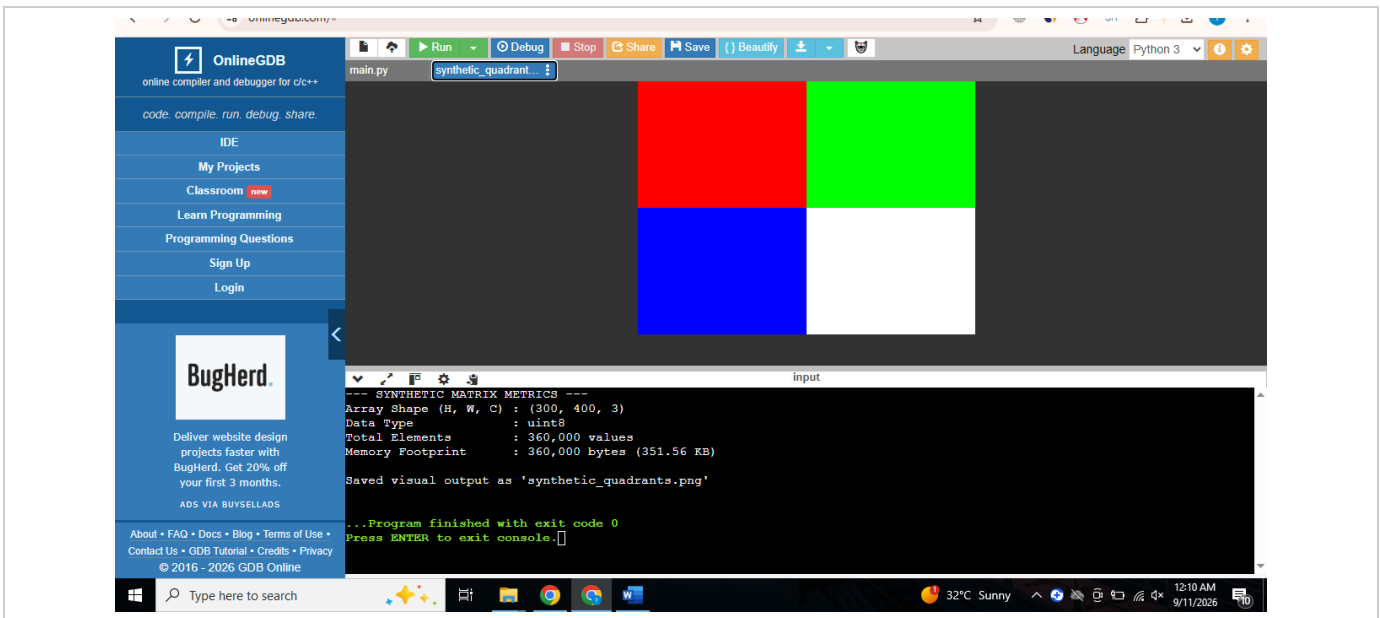


Fig 2.2 — Rendered 300×400 image: four quadrants filled via NumPy slicing (Red, Green, Blue, White).

Results Summary

Metric	Value
Array Shape (H, W, C)	(300, 400, 3)
Data Type	uint8
Total Elements	360,000 values
Memory Footprint	360,000 bytes (351.56 KB)

Observation: Slicing the array with `img[row_start:row_end, col_start:col_end] = [R,G,B]` broadcasts the 3-value colour across every pixel in that rectangular region, which is why each 150×200 quadrant is filled in a single assignment instead of a nested loop. Since dtype is uint8, each of the 360,000 channel values occupies exactly 1 byte, so element count and byte count are numerically identical.

3. Task 3: Channel Slicing & Isolation

Objective: Load an image, extract individual 2D channel intensity grids, and isolate single-channel color views.

Source Code

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import os

def load_image(path="sample.jpg"):
    if os.path.exists(path):
        from PIL import Image
        return np.array(Image.open(path).convert("RGB"))
    else:
        H, W = 300, 400
        img = np.zeros((H, W, 3), dtype=np.uint8)
```

```

img[:, :, 0] = np.linspace(0, 255, W, dtype=np.uint8)
img[:, :, 1] = np.linspace(0, 255, H, dtype=np.uint8).reshape(-1, 1)
img[:, :, 2] = 128
return img

def main():
    img = load_image("sample.jpg")

    red_channel = img[:, :, 0]
    green_channel = img[:, :, 1]
    blue_channel = img[:, :, 2]

    red_only = np.zeros_like(img)
    red_only[:, :, 0] = red_channel

    green_only = np.zeros_like(img)
    green_only[:, :, 1] = green_channel

    blue_only = np.zeros_like(img)
    blue_only[:, :, 2] = blue_channel

    fig, axes = plt.subplots(2, 3, figsize=(12, 7))
    axes[0, 0].imshow(red_only); axes[0, 0].set_title("Red Only")
    axes[0, 1].imshow(green_only); axes[0, 1].set_title("Green Only")
    axes[0, 2].imshow(blue_only); axes[0, 2].set_title("Blue Only")
    axes[1, 0].imshow(red_channel, cmap="gray"); axes[1, 0].set_title("Red Intensity")
    axes[1, 1].imshow(green_channel, cmap="gray"); axes[1, 1].set_title("Green Intensity")
    axes[1, 2].imshow(blue_channel, cmap="gray"); axes[1, 2].set_title("Blue Intensity")
    plt.tight_layout()
    plt.savefig("channel_extraction.png", dpi=150)

    print("--- CHANNEL EXTRACTION SUMMARY ---")
    print(f"Original Image Shape : {img.shape}")
    print(f"Red Channel 2D Shape : {red_channel.shape} | Mean Intensity: {red_channel.mean():.2f}")
    print(f"Green Channel 2D Shape: {green_channel.shape} | Mean Intensity: {green_channel.mean():.2f}")
    print(f"Blue Channel 2D Shape : {blue_channel.shape} | Mean Intensity: {blue_channel.mean():.2f}")

if __name__ == "__main__":
    main()

```

Execution on Online GDB

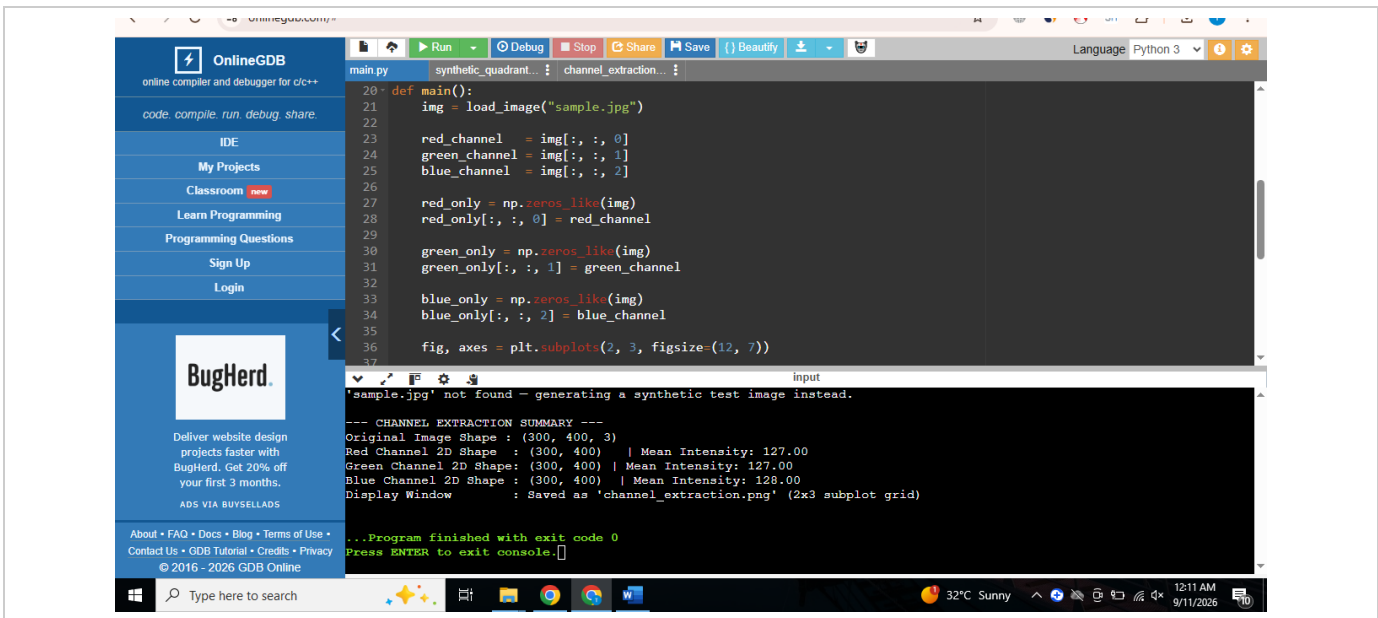


Fig 3.1 — Console output: per-channel shape and mean intensity (fallback synthetic image, since sample.jpg was not uploaded).

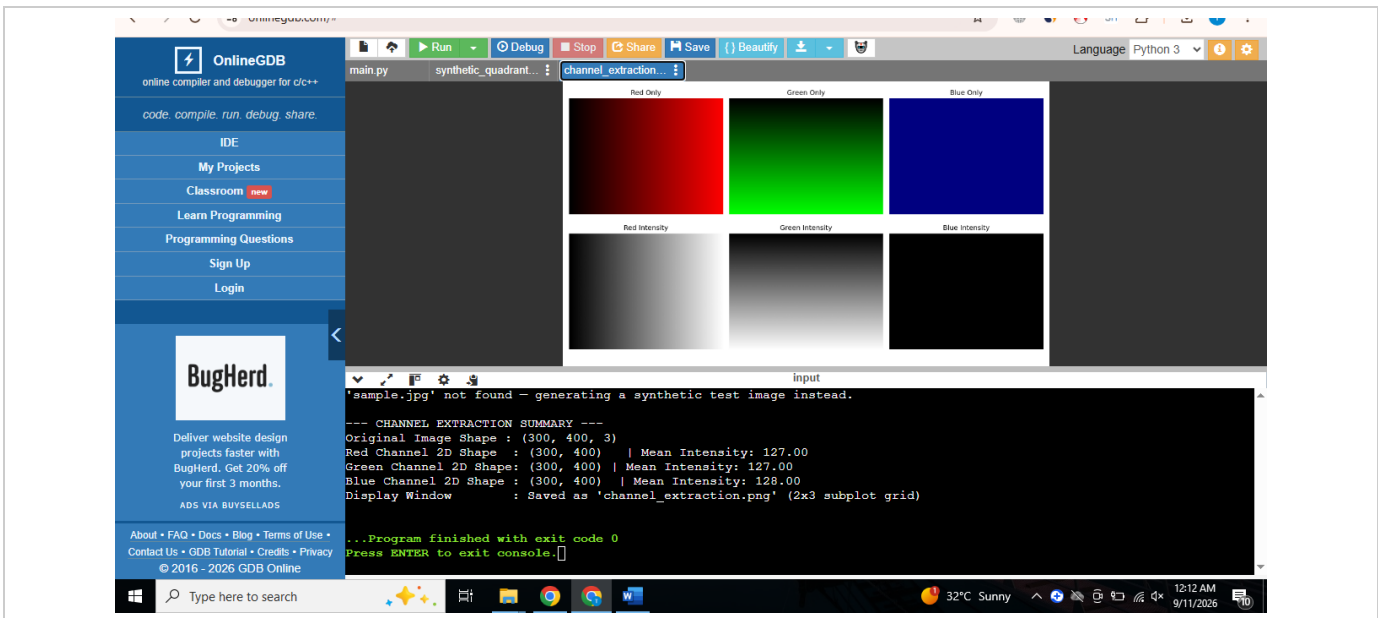


Fig 3.2 — 2x3 subplot grid: top row shows isolated R/G/B colour views, bottom row shows grayscale intensity maps.

Results Summary

Channel	2D Shape	Mean Intensity
Red	(300, 400)	127.00
Green	(300, 400)	127.00
Blue	(300, 400)	128.00

Observation: `img[:, :, 0]`, `img[:, :, 1]`, and `img[:, :, 2]` slice along Axis 2 to isolate each colour plane as a 2D array. Placing that plane back into an otherwise-zero array (`np.zeros_like`) reconstructs a full-colour image that only shows one channel's contribution, which is why the 'Red Only' panel appears as shades of red rather than grayscale.

4. Task 4: Spatial Downsampling & Pixelation via Striding

Objective: Perform spatial downsampling directly in NumPy using slice steps.

Source Code

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import os

def load_image(path="sample.jpg"):
    if os.path.exists(path):
        from PIL import Image
        return np.array(Image.open(path).convert("RGB"))
    else:
        H, W = 1080, 1920
        img = np.zeros((H, W, 3), dtype=np.uint8)
        block = 40
        for i in range(0, H, block * 2):
            for j in range(0, W, block * 2):
                img[i:i+block, j:j+block] = [255, 100, 50]
                img[i+block:i+2*block, j+block:j+2*block] = [50, 100, 255]
        return img

def main():
    N = 8
    img = load_image("sample.jpg")
    original_shape = img.shape
    original_bytes = img.nbytes

    downsampled = img[::N, ::N, :]
    down_shape = downsampled.shape
    down_bytes = downsampled.nbytes

    reexpanded = np.repeat(downsampled, N, axis=0)
    reexpanded = np.repeat(reexpanded, N, axis=1)
    reexpanded = reexpanded[:original_shape[0], :original_shape[1], :]

    dim_reduction_pct = (1 - 1 / N) * 100
    mem_savings_pct = (1 - down_bytes / original_bytes) * 100

    print(f"--- DOWNSAMPLING ANALYSIS (N = {N}) ---")
    print(f"Original Shape      : {original_shape} | Memory: {original_bytes:,} bytes")
    print(f"Downsampled Shape   : {down_shape} | Memory: {down_bytes:,} bytes")
    print(f"Dimension Reduction: {dim_reduction_pct:.2f}% reduction per axis")
    print(f"Memory Savings      : {mem_savings_pct:.2f}% data reduction")

    fig, axes = plt.subplots(1, 2, figsize=(10, 5))
    axes[0].imshow(img); axes[0].set_title("Original")
    axes[1].imshow(reexpanded); axes[1].set_title(f"Pixelated (N={N})")
    plt.tight_layout()
    plt.savefig("downsampling_comparison.png", dpi=150)

if __name__ == "__main__":
    main()
```

Execution on Online GDB

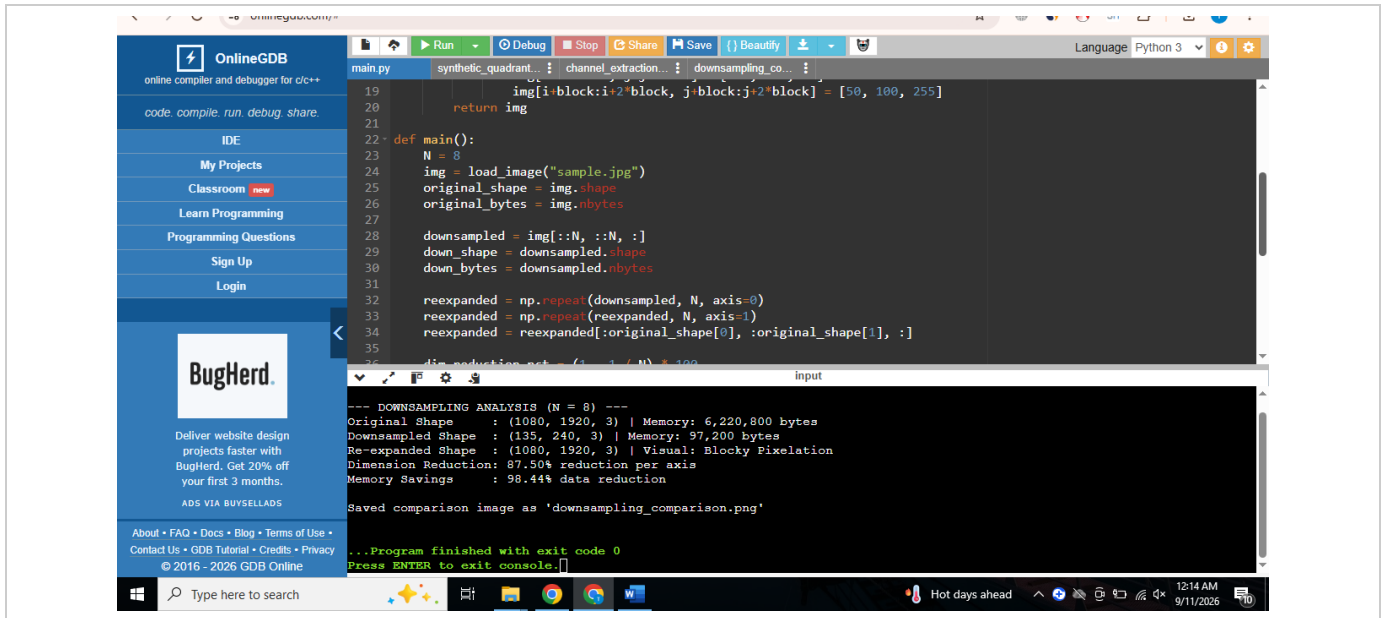


Fig 4.1 — Console output: original vs. downsampled vs. re-expanded shapes and memory statistics (N = 8).

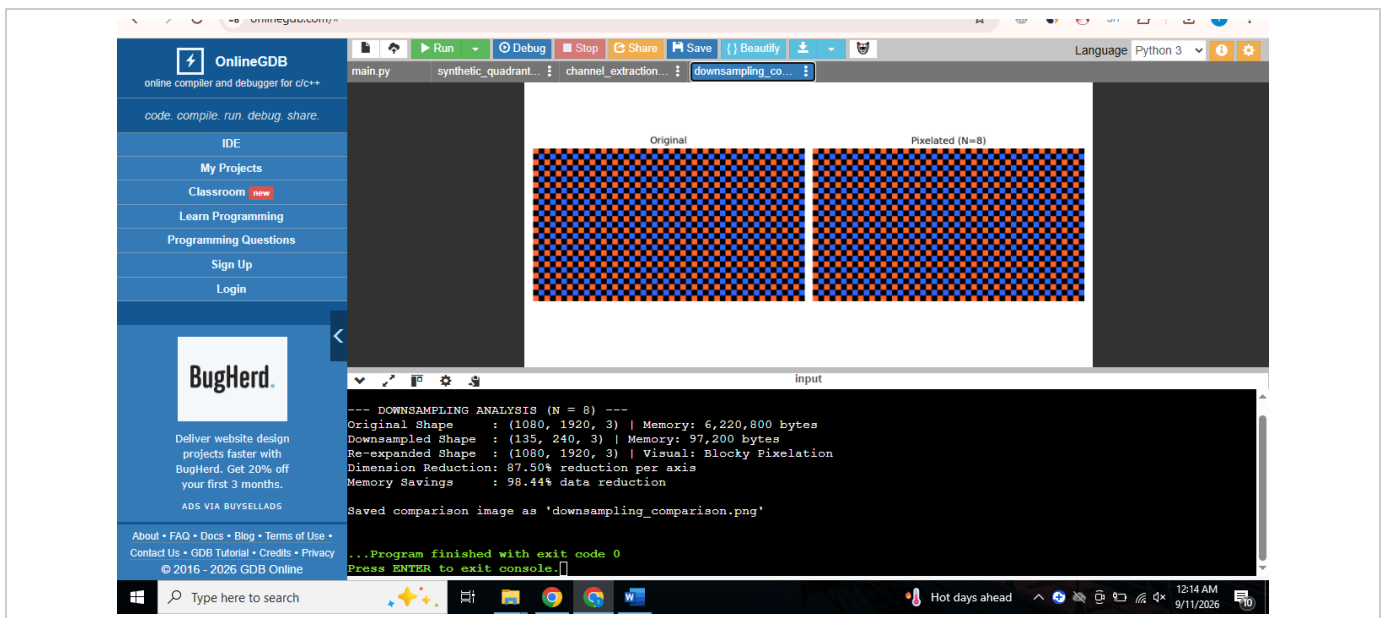


Fig 4.2 — Side-by-side comparison: original checkerboard test image vs. its blocky, pixelated re-expansion.

Results Summary

Metric	Original	Downsampled (N=8)
Shape	(1080, 1920, 3)	(135, 240, 3)
Memory	6,220,800 bytes	97,200 bytes
Dimension Reduction	—	87.50% per axis
Memory Savings	—	98.44%

Observation: `img[::N, ::N, :]` keeps only every 8th row and column, shrinking each spatial dimension to 1/8th of its original size (87.5% reduction) while leaving the channel axis untouched. Because memory scales with

$H \times W \times C$, the byte savings compound to roughly $1/64$ th of the original (98.44%), matching $(1/8)^2$ for the two downsampled axes. `np.repeat()` then duplicates each retained pixel N times along both axes to restore the original canvas size, producing the visible blocky/pixelated artifact seen in Fig 4.2.

Short Answer Question

Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

A standard RGB image stores 3 channels per pixel (Red, Green, Blue), each held in 1 byte (uint8) — 3 bytes per pixel in total. Adding an Alpha channel for transparency introduces a 4th byte per pixel, giving 4 bytes per pixel for RGBA. The memory footprint therefore scales with the channel count C in the (H, W, C) array, and 4 bytes is $4/3 \approx 1.333\times$ the original 3 bytes — i.e. a 33% increase. The extra channel doesn't touch the spatial resolution ($H \times W$ stays the same); it purely adds one more parallel plane along Axis 2, which is why the increase is a fixed one-third rather than something that depends on image size.